📘 Day 19 – Quick Sort (Complete Notes)

🚀 Introduction

Quick Sort ek **Divide and Conquer** sorting algorithm hai jo array ko efficiently sort karta hai.

👉 Ye practical scenarios me sabse fast sorting algorithms me se ek mana jata hai.

💡 Basic Idea

👉 Ek element choose karo → **Pivot**

👉 Array ko 2 parts me divide karo:

- Left side → Pivot se chhote elements
- Right side → Pivot se bade elements

👉 Phir recursively dono parts ko sort karo

🧠 Simple Line

👉 "Pick pivot → Place it correctly → Repeat"

🔁 Working Example

Array:
[6, 3, 9, 5, 2, 8]

Step 1: Pivot = 8
Left: [6, 3, 5, 2] | Pivot: 8 | Right: [9]

Step 2: Left part sort
[3, 2] 5 [6]

Final Sorted Array:
[2, 3, 5, 6, 8, 9]

⚙️ Algorithm Steps

1. Pivot select karo (usually last element)

2. Partition process apply karo
3. Pivot ko correct position pe rakho
4. Left aur Right subarray par recursion call karo

🔥 **Partition Concept**

Partition ka kaam:

- Pivot ko uski correct position pe rakhna
- Left side → Smaller elements
- Right side → Greater elements

💻 Java Code

```java
public class QuickSort {

    public static void quickSort(int[] arr, int low, int high){

        if(low < high){

            int pivotIndex = partition(arr, low, high);

            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }

    public static int partition(int[] arr, int low, int high){

        int pivot = arr[high];
        int i = low - 1;

        for(int j = low; j < high; j++){

            if(arr[j] < pivot){
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i+1];
        arr[i+1] = arr[high];
        arr[high] = temp;

        return i+1;
    }

    public static void main(String[] args){

        int[] arr = {6,3,9,5,2,8};

        quickSort(arr, 0, arr.length - 1);

        for(int num : arr){
            System.out.print(num + " ");
        }
    }
}

```

✅ Output

2 3 5 6 8 9

🔽 Descending Order

Condition change karo:

if(arr[j] > pivot)

⚡ Time Complexity

Case	Time Complexity
Best Case	O(n log n)
Average	O(n log n)
Worst Case	$O(n^2)$

⚠️ Worst Case Kab?

👉 Jab array already sorted ho aur pivot last element ho

Example:
[1, 2, 3, 4, 5]

🧠 Space Complexity

👉 O(log n)

Reason: Recursion stack

⚔️ Quick Sort vs Merge Sort

Feature	Merge Sort	Quick Sort
Time	O(n log n)	O(n log n)
Space	O(n)	O(log n)
Stable	Yes	No
Practical	Medium	Best

🔥 Optimization (Important)

👉 Random Pivot use karo:

- Worst case avoid hota hai

- Performance improve hoti hai

🎯 Important Interview Questions

1. Kya Quick Sort stable hai?
   👉 No
2. Kya ye in-place hai?
   👉 Yes
3. Quick Sort fast kyun hai?
   👉 Due to in-place sorting and better cache performance

🚀 Practice Problems

1. Quick Sort implement karo
2. Descending order me sort karo
3. Random pivot implement karo
4. Kth smallest element (Quick Select)

🏁 Conclusion

Quick Sort ek powerful sorting algorithm hai jo pivot ke around partition karke recursively array ko sort karta hai.

👉 Ye practical scenarios me fastest sorting algorithms me se ek hai.

🔥 Golden Line

👉 “Quick Sort pivot ke around divide karke fast sorting karta hai”